ARTICLE | Article in Press

A dynamic explainable AI framework for real-time intrusion detection and automated alert prioritization in security operation centers

Muhammad Usama Nazir · Mohammed Alsuhaibani · Asri bin Ngadi

Received: 4 May 2026 / Accepted: 16 September 2026
© The Author(s) 2026

Abstract

Security Operation Center (SOC) analysts face 50–100 security alerts per hour, leading to cognitive fatigue and delayed incident response. Existing intrusion detection systems (IDS) suffer from high false-positive rates and opaque black-box architectures that erode analyst trust and slow triage decisions. This paper proposes the *Dynamic Explainable Framework* (DEF), which couples a machine-learning detection backbone with inference-time SHAP and LIME explanations and an explanation-aware alert prioritization layer. DEF is evaluated on two benchmarks under an identical protocol: a stratified 60,900-flow subset of CICIoT2023 (eight classes) and a deduplicated 60,889-flow subset of UNSW-NB15 (ten classes). The XGBoost detection backbone attains **94.05 ± 0.08%** accuracy with **88.49 ± 0.13%** macro F1 at a **1.29%** false-positive rate on CICIoT2023, and **88.26 ± 0.07%** accuracy with **68.22 ± 0.22%** macro F1 at a **1.25%** false-positive rate on UNSW-NB15; false-positive rate and AUC-ROC therefore transfer essentially unchanged across domains (0.99 and 0.98). Measured against the alerts the detector actually generates (true plus false positives, 2,849 on CICIoT2023 and 2,912 on UNSW-NB15) rather than against all inspected flows, the prioritization layer removes roughly half of the residual false positives at the operating point (91 of 184 on CICIoT2023) while raising macro precision by 2.8 points, at a bounded cost in recall. The residual recall cost concentrates on stealthy, low-signature attack classes, and a per-class threshold analysis bounds the safe operating range. SHAP explanations are generated at **1.6 ms** per alert (fidelity $\rho = 0.79$ against permutation importance) and LIME at **144.4 ms** (top-10 feature stability 0.62), with a combined detection-plus-attribution latency of **3.17 ms** per alert and a sustained throughput of **315 alerts per second** on commodity CPU hardware. Transformer-based temporal and graph-based topology-context modules were additionally implemented and evaluated; their fusion does not surpass the tabular backbone on either benchmark, a negative result that we report and analyze. Comparing the two datasets isolates its cause: roughly **half** (51%) of the fusion's shortfall on CICIoT2023 is attributable to that benchmark's omission of per-flow host and timestamp identifiers, which forces proxy sequence and graph construction. The framework is designed and profiled for real-time operation on benchmark traffic traces; validation in a live SOC deployment remains future work.

Keywords Intrusion detection system · Deep learning · Explainable AI · SHAP · LIME · Alert prioritization · Security operation center · XGBoost · Cross-dataset generalization · Cybersecurity

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-72373-6

Published online: 24 September 2026

Introduction

The exponential proliferation of networked and IoT devices has fundamentally altered the cybersecurity threat landscape. Modern enterprise networks generate traffic volumes that no human team can inspect manually, and Security Operation Centers (SOCs) are increasingly reliant on automated alerting systems. However, automation without interpretability creates a new problem: SOC analysts receive 50–100 security alerts per hour from automated IDS, of which a large proportion are false positives that consume analyst attention without representing real threats[1,28]. Consider a banking SOC facing a credential-stuffing campaign, or an ISP SOC during a volumetric DDoS: in both cases the operational bottleneck is not detection per se, but the analyst's ability to triage a flood of undifferentiated, unexplained alerts.

Traditional signature-based IDS such as Snort[26] and Suricata[27] match traffic against known attack signatures with high precision for documented threats, but are blind to zero-day attacks and novel evasion techniques[2,22]. Machine learning-based IDS have substantially improved anomaly detection capability, with models such as Random Forest[23], XGBoost[24], and deep architectures[12] achieving strong results on standard benchmarks[11]. However, these models share a critical weakness: they are black boxes. An analyst who receives an alert from a black-box model cannot verify whether the prediction is correct, cannot explain it to a stakeholder, and cannot develop intuition about what network conditions trigger false positives. This lack of interpretability is a recognized barrier to operational IDS adoption[9,10].

Explainable AI (XAI) addresses the transparency problem directly. SHapley Additive exPlanations (SHAP)[6] and Local Interpretable Model-Agnostic Explanations (LIME)[8] have emerged as the dominant post-hoc explanation techniques in cybersecurity, providing feature-level attributions that explain individual detection decisions[3,4]. However, three gaps persist in the literature. First, most XAI-IDS studies apply SHAP or LIME offline, after training, and do not quantify explanation latency or quality at inference time[4]. Second, the joint effect of temporal and topological modeling on tabular IDS benchmarks is rarely evaluated systematically against strong tree-ensemble baselines[15,16]. Third, reducing the volume of alerts presented to analysts — the primary driver of SOC workload — is rarely addressed jointly with explainability; where prioritization is addressed, as in the severity-driven mitigation layer of Sumathi[5], the explanation itself is not used as prioritization evidence and explanation quality is not quantified.

This paper addresses these gaps through the **Dynamic Explainable Framework (DEF)**, which makes the following contributions:

- A unified SOC-oriented pipeline that couples a strong tabular detection backbone (XGBoost) with inference-time SHAP and LIME explanations, quantified not only by latency (1.6 ms and 144.4 ms per alert respectively) but also by fidelity and stability metrics.
- An explanation-aware alert prioritization layer whose score combines detection confidence with two auxiliary terms. At matched alert volume the detection-confidence term alone accounts for the layer's full benefit; the SHAP-attribution and topology-context terms are retained for their auditability value rather than for a measurable ranking gain, a point we establish by explicit ablation.
- A systematic, statistically validated evaluation (five seeded runs, paired significance tests) of Transformer-based temporal and graph-inspired topology modules on CICIoT2023, including an honest negative result: on this benchmark, whose released features lack host and timestamp identifiers, the fusion of these modules does not surpass the tabular backbone.
- A per-class analysis of how alert suppression affects rare and stealthy attack classes, with actionable guidance on threshold selection for rare-class-sensitive deployments.
- A cross-dataset evaluation on CICIoT2023 and UNSW-NB15 under one protocol, which establishes that the false-positive rate and the alert reduction rate transfer across domains, and which uses the difference between the two benchmarks to quantify how much of the negative fusion result is caused by absent host and timestamp identifiers rather than by the architecture.

The unifying element of these contributions is that the explanation is an *input* to the pipeline rather than an artifact attached to it: attribution magnitude is a term of the alert-priority score (Section "Alert prioritization layer"), computed for every alert at inference time.

The remainder of this paper is organized as follows. Section "Related work" surveys related work. Section "Proposed methodology: dynamic explainable framework" presents the DEF architecture and mathematical formulations. Section "Experimental setup" describes the experimental setup. Section "Results" presents and analyzes the results. Section "Discussion" discusses implications and limitations. Section "Conclusion" concludes.

Related work

Machine learning-based intrusion detection

Machine learning approaches to intrusion detection have advanced substantially since the introduction of the KDD Cup 1999 dataset[17]. Classical methods provided the first generation of adaptive IDS, but their dependence on handcrafted features limited generalization[22]. The availability of richer benchmark datasets including NSL-KDD[17], UNSW-NB15[20], CIC-IDS-2017/2018[18], and most recently CICIoT2023[19] enabled systematic comparison of more sophisticated models.

Ensemble methods, particularly Random Forest[23] and XGBoost[24], have consistently achieved high accuracy on structured network traffic data. Talukder et al.[11] demonstrated that hybrid ensembles achieve F1-scores above 97% on CIC-IDS-2017 but observed that this performance did not translate to interpretable reasoning for analysts. Deep learning approaches including LSTMs and autoencoders[12] have extended detection capability to more complex patterns, though their computational cost and opacity create deployment challenges in latency-sensitive SOC environments. Sommer and Paxson[28] identified the gap between benchmark accuracy and operational usefulness — including alert fatigue — as a fundamental challenge for ML-based IDS over a decade ago; the problem has intensified with traffic growth.

Explainable AI techniques for IDS

Arrieta et al.[9] provided a taxonomy of XAI techniques categorizing ante-hoc (inherently interpretable) and post-hoc methods, and Doshi-Velez and Kim[10] formalized evaluation criteria for interpretability. Among post-hoc approaches, SHAP[6] — with its efficient TreeExplainer variant for tree ensembles[7] — and LIME[8] are the most widely applied in cybersecurity.

Gaspar et al.[4] compared SHAP and LIME applied to a Multi-Layer Perceptron IDS on CIC-IDS-2017, finding that SHAP produces more globally consistent feature attributions while LIME provides faster instance-specific explanations. Mohale and Obagbuwa[3] reviewed XAI-enhanced IDS and reported evidence that explanations improve analyst trust and support triage. These studies shaped two DEF design decisions: the use of TreeExplainer for low-latency global and local attributions, and the inclusion of explanation-quality metrics (fidelity and stability) beyond raw execution time.

Closest in operational intent to the present work is the explainable hybrid framework of Sumathi[5] for software-defined networks, which combines SHAP-guided feature preprocessing with a temporal-convolutional and gradient-boosted ensemble, and couples detection to a mitigation layer that selects countermeasures from an attack severity score. That design confirms the practical value of ranking alerts rather than merely classifying them. It differs from the DEF in two respects that define our contribution: the severity score is derived from the predicted class rather than from the explanation itself, so the attribution does not participate in ranking; and explanation quality is not quantified, so the fidelity and stability of the attributions on which analysts would rely remain unmeasured. The DEF instead makes the attribution magnitude a term of the priority score and reports fidelity, stability, and per-alert latency for every explanation method used.

Temporal and graph-based IDS

Transformer architectures[13] have been applied to sequential network data to capture long-range dependencies; a recent example is the Transformer-based traffic-analysis approach of Alazzam et al.[29], which detects concealed VPN-based behavior from traffic sequences and independently supports the suitability of attention mechanisms for network-security sequence modeling. Graph Neural Networks, and specifically Graph Convolutional Networks (GCNs)[14], have been applied to IDS for modeling structural relationships between hosts, enabling detection of lateral movement and botnet propagation invisible to per-flow models[15,16]. These works motivated the temporal and topology-context modules evaluated in this study; a key open question — which we address empirically — is whether such modules retain their value on benchmarks whose released features omit the host and timestamp identifiers that true sequence and graph construction require.

Research gap summary

Table 1 positions the DEF against the most closely related studies.

Stated plainly, the gap the DEF fills is the point of coupling. Existing XAI-IDS studies generate SHAP or LIME attributions as post-hoc artifacts for offline inspection, and do not measure their fidelity, stability, or per-alert cost at inference time[3,4]. Existing prioritization and mitigation layers rank alerts on detection confidence or on the severity of the predicted class, so the explanation — where one exists at all — plays no part in deciding which alerts an analyst sees[5]. To our knowledge, the DEF is the first XAI-IDS framework to make the explanation a first-class, quantified component of the SOC pipeline: SHAP and LIME attributions are computed at inference time, their fidelity and stability are measured rather than assumed, and the attribution is presented to the analyst alongside each surviving alert as directly verifiable evidence. We evaluated including the attribution magnitude as a term of the priority score and report the result honestly: by explicit ablation (Table 13) the detection-confidence term alone accounts for the layer's ranking benefit, so we retain the attribution for its auditability value rather than for a ranking gain. The novelty claimed is therefore architectural and operational — where the explanation sits in the pipeline and what it is used for — not a detection-accuracy gain over a strong tabular baseline, a distinction the component analysis of Section "Component analysis" makes explicit.

Proposed methodology: dynamic explainable framework

Framework architecture

Figure 1 illustrates the DEF pipeline. The framework operates in three stages: detection, explanation, and prioritization. The detection backbone is an XGBoost classifier over flow-level features; two auxiliary modules — a

Table 1 Comparison of related XAI-IDS studies with the proposed DEF. "Explanation quality" denotes quantitative evaluation of explanations beyond execution time (e.g., fidelity, stability). "Explanation-aware priority" denotes use of the explanation itself as ranking evidence, as distinct from ranking on the predicted class alone.

Study	Temporal model	Topology aware	Explanation-aware priority	SHAP/LIME	Explanation quality
Gaspar et al.[4]	No	No	No	Yes	No
Mohale & Obagbuwa[3]	No	No	No	Yes	No
Talukder et al.[11]	No	No	No	No	No
Lo et al.[15]	No	Yes	No	No	No
Caville et al.[16]	No	Yes	No	No	No
Alazzam et al.[29]	Yes	No	No	No	No
Sumathi[5]	Yes	No	Class-severity only	Yes	No
DEF (this work)	**Yes (eval.)**	**Yes (eval.)**	**Yes**	**Yes**	**Yes**

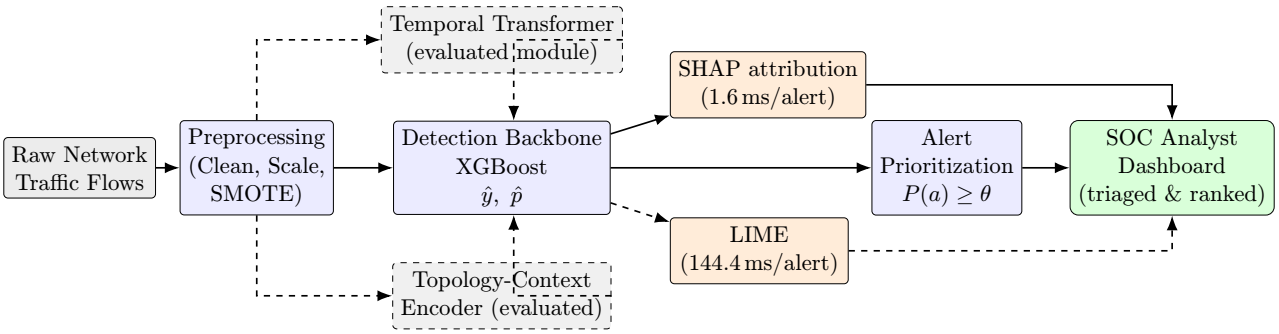


Fig. 1 DEF architecture. Solid arrows: primary flow. The priority score $P(a)$ is driven by detection confidence; SHAP and LIME attributions are computed for the alerts presented to the analyst and shown alongside them as verifiable rationale (auditability), rather than used to rank them. Dashed modules (Transformer, topology-context encoder) are evaluated candidates whose measured contribution is reported in the component analysis.

time-series Transformer and a topology-context encoder — are evaluated as candidate enhancements to the backbone and reported in the component analysis (Section "Component analysis").

Sequence construction and time-series transformer

The released CICIoT2023 feature files do not contain per-flow host addresses or timestamps. Sequences are therefore constructed as follows. The flows are *first* partitioned into training, validation and test sets; windows are then built *within* each partition, so that no window ever spans a partition boundary and no raw flow contributes to windows in more than one partition. Within a partition, flows are kept in the chronological order of the original capture files, grouped by protocol-type signature, and segmented into sliding windows of $T = 32$ consecutive flows with stride 1; a group of fewer than T flows yields a single left-padded window. Each flow is the label of *exactly one* window — the window that ends on it — so the window-level class distribution mirrors the flow-level distribution and no flow is counted more than once. The per-partition window counts by class are given in Table 3. We note explicitly that this protocol-group ordering is a proxy for true per-host sequences, a limitation revisited in Section "Limitations".

Within a window $\mathrm{X} = \{x_1, \ldots, x_T\}$, $x_t \in \mathbb{R}^d$, the multi-head self-attention mechanism[13] computes:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

with $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and sinusoidal position encodings added to preserve temporal order:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{2}$$

The mean-pooled encoder output h_T summarizes the temporal context of the window.

Topology-context encoder

Graph Convolutional Networks[14] model host communication graphs whose nodes are IP addresses and whose edges are flows, enabling detection of propagation patterns such as lateral movement[15,16]. Constructing such a graph requires per-flow host identifiers and timestamps, which the released CICIoT2023 features omit. We therefore implement a *topology-context encoder* as the closest realizable substitute: for each flow, a four-dimensional neighborhood descriptor is computed within its protocol group — (i) log group size, (ii) rolling mean and (iii) rolling standard deviation of local activity over a 64-flow neighborhood, and (iv) relative position within the

group — and passed through a two-layer perceptron to produce the context embedding $\mathrm{g} \in \mathbb{R}^{64}$. Local activity is taken as the highest-variance retained feature (Appendix A), which serves as a scalar proxy for flow intensity within the neighborhood. This descriptor plays the role that node-degree and fan-out statistics play in a host graph. The consequences of this substitution are analyzed in Sections "Component analysis" and "Limitations".

Feature fusion and classification

For the fusion configuration, the Transformer output h_T and context embedding g are concatenated and passed through a classification head:

$$\hat{y} = \mathrm{softmax}(W_c\,[\mathrm{h}_T \,\|\, \mathrm{g}] + b_c) \tag{3}$$

trained with class-weighted cross-entropy loss (inverse-frequency weights) to counter class imbalance:

$$\mathcal{L} = -\sum_{i=1}^{N}\sum_{c=1}^{C} w_c\, y_{ic} \log \hat{y}_{ic} \tag{4}$$

The tabular backbone (XGBoost[24]) is trained independently on the SMOTE-balanced training fold.

SHAP explanation module

For each alert, a Shapley value ϕ_j is computed for every feature j under the unified framework of Lundberg and Lee[6]:

$$\phi_j(f,x) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F|-|S|-1)!}{|F|!}\left[f_{S\cup\{j\}}(x_{S\cup\{j\}}) - f_S(x_S)\right] \tag{5}$$

For the tree-ensemble backbone, the TreeExplainer algorithm[7] reduces this computation to polynomial time, yielding a measured 1.6 ms per-alert explanation. For the neural fusion model, gradient-based SHAP is used instead and is substantially slower (983.9 ms per alert); this distinction is reported wherever explanation latency is discussed.

LIME explanation module

For an alert instance x, LIME fits a local surrogate by solving[8]:

$$\xi(x) = \arg\min_{g \in \mathcal{G}} \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{6}$$

where $\pi_x(z) = \exp(-D(x,z)^2/\sigma^2)$ is the proximity kernel and $\Omega(g)$ penalizes surrogate complexity. The output is a ranked list of ten feature conditions with signed influence on the classification, generated at a measured 144.4 ms per instance.

Alert prioritization layer

Every alert a is assigned a composite priority score:

$$P(a) = \alpha\,\hat{p}_{\text{attack}}(a) + \beta\,S_{\text{SHAP}}(a) + \gamma\,C_{\text{ctx}}(a), \qquad \alpha+\beta+\gamma = 1, \quad \alpha,\beta,\gamma \geq 0 \tag{7}$$

where $\hat{p}_{\text{attack}}$ is the backbone's predicted probability of a non-benign class; $S_{\text{SHAP}} = \frac{1}{10} \sum_{j \in \text{top-10}} |\phi_j|$ measures how decisively the feature evidence supports the alert; and C_{ctx} is the neighborhood-activity dispersion of the alert's protocol group taken from the topology-context descriptor of Section "Topology-context encoder", capturing how anomalous the surrounding traffic neighborhood is. Each of the three terms is min–max normalized to [0, 1] using statistics estimated on the *validation* split and then frozen; the same fixed minima and maxima are applied unchanged to the test split and would be applied to streaming traffic at deployment. No test-set statistic enters the normalization, so $P(a) \in [0, 1]$ is computed identically at evaluation and deployment time, and the weights remain directly interpretable as the relative influence of the three evidence sources.

The linear form is a deliberate design choice rather than a claim of optimality. It is monotone in each term, so an analyst can reason about why one alert outranks another; it is auditable, since the contribution of each term to a given score can be displayed alongside the alert; and it admits exhaustive tuning in a single pass over a three-parameter simplex. The decision rule is to present alert a to the analyst if and only if $P(a) \geq \theta$.

Metrics reported after prioritization are computed over the *full* evaluation set rather than over the surviving alerts. An alert with $P(a) < \theta$ is recorded as a benign (no-alert) decision,

$$\hat{y}^{\theta}(a) = \begin{cases} \arg\max_{c} \hat{p}_c(a) & \text{if } P(a) \geq \theta, \\ \text{Benign} & \text{otherwise,} \end{cases} \tag{8}$$

so that a suppressed true attack is counted as a missed detection. Precision, recall, and macro F1 after prioritization are therefore directly comparable with the corresponding unfiltered rows of Table 9, and the observed increase in precision reflects the removal of low-confidence false positives rather than a change of evaluation population.

Two deployment clarifications follow. First, suppression as defined by Eq. 8 is an *evaluation convention*, chosen to be conservative: every suppressed true attack is scored as a missed detection. In deployment, alerts with $P(a) < \theta$ need not be discarded: they can be routed to a low-priority queue that remains available for batch review and retrospective threat hunting, so the analyst-facing reduction concerns what is presented for immediate triage, not what is retained. Second, because the recall cost of suppression concentrates on the stealthy classes (Section "Suppression risk for stealthy and rare attack classes"), a deployment can impose a class-conditional threshold θ_c or a per-class presentation floor so that no attack family is reduced below an auditable level; we evaluate the single-threshold form here and identify the class-conditional form as future work.

The weights are tuned by exhaustive grid search over the simplex (step 0.1) on the validation split at a fixed screening threshold $\theta = 0.6$, maximizing the alert reduction rate subject to a macro-recall floor. A floor of 90% was not attainable on validation; the tuner therefore fell back to an 85% floor, selecting $\alpha = 0.5$, $\beta = 0.4$, $\gamma = 0.1$ (validation ARR 72.12%, macro recall 85.74%). Because prioritization can only reduce recall, an absolute recall floor is not transferable between benchmarks whose baseline recall differs. We therefore specify the floor relative to the unprioritized macro recall of the same dataset, descending through 98%, 96%, 94% and 90% of baseline until a configuration is admissible. On CICIoT2023 the 85% absolute floor reported above corresponds to 96% of the 88.6% validation baseline; the identical relative ladder is applied to UNSW-NB15, whose validation baseline macro recall is 69.14% and for which no absolute floor of 85% is attainable by any configuration.

The deployed operating threshold θ is selected on the *validation* split — as the smallest threshold whose validation macro recall stays above 98% of the unprioritized validation baseline — and then applied unchanged to test; no test-set performance is inspected in choosing it. The test sweep of Table 15 is reported only to characterize sensitivity, not to select the operating point. A sensitivity analysis around the selected weights is reported in Table 16.

Table 2 Dataset characteristics (CICIoT2023, 60,900-flow stratified subset; 70/15/15 train/validation/test split).

Class	Samples	Test set
Benign	42,000	6,300
Brute Force	2,700	405
DDoS	2,700	405
DoS	2,700	405
Mirai Botnet	2,700	405
Recon	2,700	405
Spoofing	2,700	405
Web Attack	2,700	405
Total	**60,900**	**9,135**

Table 3 Within-partition window counts by class (CICIoT2023, $T = 32$, stride 1). Windows are built inside each partition, so counts sum within a fold and no flow crosses a boundary (Editor Comment 2). The test column matches the flow-level test support of Table 2 up to the per-group left-pad of the first window, and each test flow is labelled exactly once ($n = 7{,}597$ test windows in total).

Class	Train	Val	Test
Benign	39,636*	—	5,505
Brute Force	—	—	365
DDoS	—	—	318
DoS	—	—	303
Mirai Botnet	—	—	242
Recon	—	—	276
Spoofing	—	—	265
Web Attack	—	—	323
Total	**39,636**	**7,571**	**7,597**

Experimental setup

Dataset

Experiments were conducted on a stratified subset of the CICIoT2023 dataset[19], a large-scale IoT attack benchmark released by the Canadian Institute for Cybersecurity. The 33 fine-grained attack labels were mapped to seven attack families plus Benign traffic, yielding eight classes: Benign, Brute Force, DDoS, DoS, Mirai Botnet, Recon, Spoofing, and Web Attack. A stratified random sample of **60,900 flows** was drawn from the official CSV releases and restored to the original chronological order of the capture files, capping Benign at 42,000 flows (69.0% of the subset) to mirror operational class priors and sampling 2,700 flows per attack family. Table 2 reports the class distribution.

Second benchmark: UNSW-NB15

To assess generalization beyond a single benchmark, all experiments were repeated on UNSW-NB15[20], a network-intrusion dataset generated on a testbed at the Australian Centre for Cyber Security and characterized statistically by Moustafa and Slay[21]. It was selected over the alternatives for two reasons. First, it is enterprise rather than IoT traffic, so it tests transfer across *domains* and not merely across captures. Second, and central to this study, its source CSV releases retain per-flow host addresses (`srcip`, `dstip`) and timestamps (`Stime`, `Ltime`) — precisely the fields CICIoT2023 omits. This permits construction of genuine per-host event sequences and a real host communication graph, and therefore a direct test of the explanation offered in Section "Interpretation of detection results" for the negative fusion result.

Deduplication. The four source CSV files contain 2,540,047 records, of which **480,633 (18.9%) are byte-identical duplicates**: every field, including host addresses, ports and both timestamps, is repeated. These are

therefore duplicated records rather than distinct flows that happen to share a feature vector, and under any random split they appear in both the training and test folds. We remove them before sampling, leaving 2,059,414 unique records. The effect is highly class-dependent — Generic falls from 215,481 to 25,378 records (88.2% duplicated) and DoS from 16,353 to 5,665 (65.4%), while Normal loses only 11.7% — so the commonly reported dominance of Generic among UNSW-NB15 attack traffic is substantially an artifact of duplication. Because most published UNSW-NB15 results do not deduplicate, the figures reported here are expected to fall below the literature and should be read as the more conservative estimate.

Sampling protocol. The sampling protocol matches the CICIoT2023 subset as closely as the data permit. The benign share is fixed at 69.0%, identical to CICIoT2023, because the benign/attack prior is what the false-positive rate and macro precision depend on; holding it constant is what makes the two evaluations comparable. The remaining budget is distributed across the nine attack families by water-filling: families whose pool is smaller than the per-class allowance are taken in full and their unused allowance is redistributed. Four families are exhausted this way (Analysis 2,184; Backdoor 1,983; Shellcode 1,511; Worms 171), so the subset is not uniform across attack classes. The resulting subset comprises **60,889 flows** in ten classes (Table 4), split 70/15/15 into 42,622/9,133/9,134 and expanded by SMOTE to 293,970 training samples. Preprocessing is otherwise identical to Section "Preprocessing pipeline": after removal of constant and duplicate columns, **41** features remain (against 37 for CICIoT2023), comprising 38 numerical flow features plus `proto`, `state` and `service` encoded as integer codes — the same treatment CICIoT2023 applies to `Protocol Type`.

Host addresses, ports and timestamps are held aside from the feature matrix and supplied only to the sequence and graph constructors. Host identity is excluded because the testbed comprises few hosts and would be memorized; absolute timestamps are excluded because attacks are time-clustered and would leak the label almost perfectly. Flow duration is retained, being a duration rather than an absolute time.

Preprocessing pipeline

1. **Cleaning**: Replace `Inf`/`-Inf` with `NaN`; drop rows with more than 50% missing values; fill remaining `NaN` with column medians. No rows were removed at this step on the sampled subset.
2. **Feature pruning**: Remove constant and duplicate columns. Of the 46 numerical flow features in the released feature files, seven were constant across the subset and two were exact duplicates of retained columns, leaving **37** features. All 37 are retained — no top-k truncation is applied — and are ordered by descending variance, since the topology-context descriptor of Section "Topology-context encoder" uses the highest-variance feature as its local-activity proxy. The full list is given in Appendix A.
3. **Normalization**: Min–max scaling $x' = (x - \min)/(\max - \min)$ fitted on the training fold only, preventing leakage.
4. **Split**: Stratified 70%/15%/15% train/validation/test ($n = 42{,}630$/9, 135/9, 135).

Table 4 UNSW-NB15 subset (60,889 flows, ten classes; 70/15/15 split). Benign share is matched to the CICIoT2023 subset at 69.0%. Four attack families are size-limited by the deduplicated corpus and are taken in full.

Class	Samples	Test set	Note
Normal	41,995	6,299	
Reconnaissance	2,609	392	
Generic	2,609	392	
Fuzzers	2,609	392	
Exploits	2,609	392	
DoS	2,609	392	
Analysis	2,184	327	pool exhausted
Backdoor	1,983	297	pool exhausted
Shellcode	1,511	226	pool exhausted
Worms	171	25	pool exhausted
Total	**60,889**	**9,134**	

5. **Class imbalance**: SMOTE[25] oversampling applied to the training fold only, expanding it from 42,630 to 235,200 samples (eight classes balanced to the majority count of 29,400); the neural models use inverse-frequency class weighting instead, evaluated as the corresponding imbalance treatment for sequence inputs.

Baseline models

The evaluated configurations are: (i) **Random Forest**[23] (200 trees, depth 20); (ii) **XGBoost**[24] (the DEF backbone); (iii) **LSTM** (2 layers, hidden 128) as the standard sequential deep baseline[12]; (iv) the **Transformer** module alone; (v) the **topology-context encoder** alone; and (vi) the **Transformer–context fusion** (Eq. 3). All ML models are trained on identical splits with identical preprocessing and the same five random seeds; signature-based systems such as Snort[26] are discussed qualitatively in Section "Related work" but excluded from the quantitative comparison because no published Snort evaluation exists for this CICIoT2023 subset and an unmatched external figure would not constitute a fair like-for-like baseline.

Implementation details

All experiments use Python 3.11. **XGBoost** 2.x: n_estimators = 300, max_depth = 6, learning rate = 0.1, subsample = 0.8, tree_method = hist. **Random Forest** (scikit-learn): n_estimators = 200, max_depth = 20. **Transformer** (PyTorch 2.x): 2 encoder layers, 4 attention heads, model dimension $d = 128$, feed-forward dimension 256, dropout 0.1, window $T = 32$, stride 1, sinusoidal positional encoding. **Topology-context encoder**: two-layer MLP, hidden width 64. **Fusion head**: concatenation followed by a 128-unit dense layer and softmax. **LSTM**: 2 layers, hidden 128. All neural models: Adam, learning rate 10^{-3}, batch size 256, up to 50 epochs with early stopping (patience 10) on validation macro F1, class-weighted cross-entropy. Every configuration is trained with five random seeds $\{1, 7, 13, 42, 101\}$; tables report mean ± standard deviation. **XAI**: SHAP TreeExplainer for the backbone, GradientExplainer for the fusion model; LIME LimeTabularExplainer with 10 features. Hardware: commodity CPU (all runtime figures in Section "Runtime and resource profile" are CPU-only).

Evaluation metrics

Accuracy; macro precision, recall, and F1; macro-averaged false-positive rate (FPR); macro one-vs-rest AUC-ROC; Alert Reduction Rate $\mathrm{ARR} = 1 - |\mathrm{Alerts}_{\mathrm{presented}}|/|\mathrm{Alerts}_{\mathrm{total}}|$; SHAP and LIME per-alert explanation time; explanation fidelity (Spearman correlation between mean $|\phi_j|$ and permutation importance); and LIME stability (mean pairwise Jaccard similarity of top-10 feature sets over five repeated explanations of each of 30 instances). Statistical significance is assessed with paired t-tests over run-level macro F1 across the five seeds, complemented by Wilcoxon signed-rank tests (whose minimum attainable p-value at $n = 5$ is 0.0625).

Results

Overall detection performance

Table 5 presents detection performance on the CICIoT2023 test set over five seeded runs. The XGBoost backbone achieves the best results on every metric: 94.05% accuracy, 88.49% macro F1, 1.29% FPR, and 0.99 AUC. The Transformer–context fusion reaches 70.35% macro F1 — significantly below the backbone (paired t-test on run-level macro F1, $p = 6.6 \times 10^{-7}$) — and neither auxiliary module closes the gap individually. Section "Interpretation of detection results" analyzes why.

Table 5 Detection performance on the CICIoT2023 test set; mean ± standard deviation over five seeds $\{1, 7, 13, 42, 101\}$. Best results in **bold**. Tabular models (Random Forest, XGBoost) are evaluated at flow level (n = 9,135); sequence and context models are evaluated at window level (n = 7,597), since each of their predictions is made for the final flow of a 32-flow window. XGBoost significantly outperforms every other configuration on run-level macro F1 (paired t-test, all $p < 6 \times 10^{-6}$; Wilcoxon signed-rank $p = 0.0625$ throughout, the minimum attainable at $n = 5$).

Model	Acc (%)	Prec (%)	Rec (%)	F1 (%)	FPR (%)	AUC
Random Forest	92.52 ± 0.04	85.10 ± 0.13	88.75 ± 0.12	86.60 ± 0.12	1.33	0.99
LSTM	76.34 ± 1.02	63.31 ± 1.51	73.08 ± 0.97	64.03 ± 0.81	3.69	0.96
Transformer	78.39 ± 2.52	64.66 ± 2.54	73.80 ± 0.78	65.72 ± 1.62	3.54	0.97
Context encoder	61.71 ± 5.14	40.57 ± 2.87	48.93 ± 2.71	41.06 ± 3.29	5.50	0.91
Transformer–context fusion	82.64 ± 1.16	68.09 ± 0.86	78.15 ± 0.75	70.35 ± 0.85	2.69	0.98
XGBoost (DEF backbone)	**94.05 ± 0.08**	**88.59 ± 0.21**	**88.57 ± 0.08**	**88.49 ± 0.13**	**1.29**	**0.99**

Table 6 Cross-dataset detection performance; mean ± standard deviation over the same five seeds $\{1, 7, 13, 42, 101\}$. CICIoT2023 has eight classes, UNSW-NB15 ten. Absolute macro F1 is therefore not directly comparable between columns, whereas FPR, AUC and the relative ordering of models are.

Dataset	Model	Acc (%)	Macro F1 (%)	FPR (%)	AUC
CICIoT2023	Random Forest	92.52 ± 0.04	86.60 ± 0.12	1.33	0.99
	XGBoost	**94.05 ± 0.08**	**88.49 ± 0.13**	**1.29**	**0.99**
UNSW-NB15	Random Forest	88.30 ± 0.04	67.70 ± 0.24	1.22	0.98
	XGBoost	**88.26 ± 0.07**	**68.22 ± 0.22**	**1.25**	**0.98**

Cross-dataset generalization

Table 6 reports the detection backbone and the Random Forest baseline on both benchmarks under identical hyperparameters, identical seeds and the same implementation.

Three observations follow.

First, **the operationally decisive properties transfer**. The false-positive rate is essentially unchanged across benchmarks (1.29% → 1.25%) and macro AUC falls by only 0.01 (0.99 → 0.98). Since analyst workload is driven by false alarms and the prioritization layer depends on well-ordered confidence scores rather than on absolute accuracy, these are the two quantities whose stability matters most for the framework's central claim.

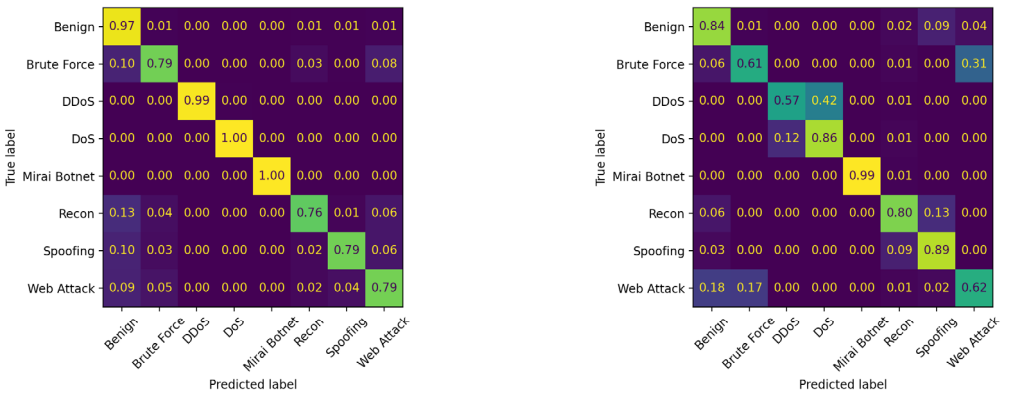
Second, **the backbone choice replicates**. XGBoost again outperforms Random Forest on macro F1 (68.22% vs. 67.70%, paired t-test $p = 1.5 \times 10^{-2}$), as it did on CICIoT2023 (88.49% vs. 86.60%). The selection of a gradient-boosted tree ensemble as the detection backbone is therefore not an artifact of one benchmark.

Third, **macro F1 falls by 20.3 points, and the drop is attributable to three specific classes rather than to degraded detection**. Table 7 gives the per-class breakdown. Normal traffic is classified almost perfectly (precision 0.998, F1 0.992), and six attack families reach F1 between 0.67 and 0.89. The macro average is depressed by Backdoor (0.249), Analysis (0.380) and DoS (0.457). This reflects the UNSW-NB15 label taxonomy rather than a modelling failure: Analysis, Backdoor and DoS are distinguished from Exploits by attacker *intent* rather than by observable flow behaviour, so a flow-level classifier has no evidence on which to separate them. Excluding those three, macro F1 over the remaining seven classes is **0.819**, against 0.885 on CICIoT2023 — a 6.6-point decline consistent with a change of both domain and class cardinality.

Notably, the rarest class is handled well: Worms, at 171 records in the entire deduplicated corpus and 25 in the test fold (0.3% prevalence), attains F1 0.680, and Shellcode reaches 0.849 with recall 0.956. The imbalance treatment therefore holds under conditions considerably more extreme than CICIoT2023 presents, where the smallest class contributes 405 test flows.

Table 7 Per-class results for the XGBoost backbone on UNSW-NB15 (seed 42, $n = 9{,}134$). Backdoor, Analysis and DoS overlap semantically with Exploits in the UNSW-NB15 taxonomy and account for essentially the whole macro-F1 deficit; the mean F1 of the remaining seven classes is 0.819.

Class	Precision	Recall	F1	Support
Normal	0.998	0.986	0.992	6,299
Generic	0.940	0.844	0.890	392
Reconnaissance	0.904	0.837	0.869	392
Shellcode	0.763	0.956	0.849	226
Fuzzers	0.759	0.804	0.781	392
Worms	0.680	0.680	0.680	25
Exploits	0.691	0.651	0.670	392
DoS	0.578	0.378	0.457	392
Analysis	0.313	0.483	0.380	327
Backdoor	0.232	0.269	0.249	297
Accuracy	0.882			9,134
Macro avg	0.686	0.689	0.682	9,134
Weighted avg	0.894	0.882	0.886	9,134



(a) XGBoost backbone (b) Transformer–context fusion

Fig. 2 Normalized confusion matrices on the CICIoT2023 test set for (**a**) the XGBoost detection backbone and (**b**) the Transformer–context fusion. The backbone shows stronger diagonal dominance (Benign 0.97 vs. 0.84; DDoS 0.99 vs. 0.57); both models detect Mirai Botnet near-perfectly.

Per-class detection performance

Figure 2 shows normalized confusion matrices for the backbone and the fusion model. The backbone achieves per-class recall of 0.97 (Benign), 0.99 (DDoS), 1.00 (DoS), 1.00 (Mirai Botnet), 0.79 (Brute Force), 0.76 (Recon), 0.79 (Spoofing), and 0.79 (Web Attack): volumetric and botnet attacks are detected near-perfectly, while stealthy, low-signature classes (Recon, Spoofing, Brute Force, Web Attack) are harder, with their residual errors flowing chiefly into the Benign class. Table 8 reports the fusion model's per-class results, whose most visible weakness is DDoS/DoS confusion (0.57 diagonal for DDoS versus 0.99 for the backbone).

Component analysis

Table 9 isolates the measured contribution of each DEF component.

Four findings emerge. First, the explanation layer leaves detection metrics unchanged, as expected for post-hoc methods: its value is transparency, quantified separately in Section "XAI explanation quality and efficiency". Second, the two auxiliary modules do combine productively with one another — the fusion improves significantly on the Transformer alone (70.35% vs. 65.72% macro F1, paired t-test $p = 3.2 \times 10^{-3}$) and on the context encoder alone ($p = 3.4 \times 10^{-5}$) — so the fusion is a coherent architecture rather than an arbitrary concatenation. Third, and

Table 8 Per-class results for the Transformer–context fusion model (seed 42; window-level evaluation, $n = 7{,}597$). The fusion detects Mirai Botnet near-perfectly but confuses DDoS with DoS and over-predicts the stealthy classes, explaining its lower macro F1 relative to the backbone.

Class	Precision	Recall	F1	Support
Benign	0.98	0.84	0.90	5,505
Brute Force	0.66	0.61	0.63	365
DDoS	0.83	0.57	0.67	318
DoS	0.66	0.86	0.75	303
Mirai Botnet	1.00	0.99	1.00	242
Recon	0.57	0.80	0.67	276
Spoofing	0.30	0.89	0.45	265
Web Attack	0.39	0.62	0.48	323
Macro avg	0.67	0.77	0.69	7,597
Weighted avg	0.88	0.81	0.83	7,597

Table 9 Component analysis on CICIoT2023 (five seeds, mean values). The temporal and topology-context modules, alone and fused, do not match the tabular backbone on this dataset; the prioritization layer removes roughly half the residual false positives (91 of 184) at the operating point, raising precision by 2.8 points at a 1.1-point recall cost; the alert-based reduction is 5.8%.

Configuration	Macro F1 (%)	Precision (%)	FPR (%)	ARR (%)
Context encoder only	41.06	40.57	5.50	—
LSTM (sequential baseline)	64.03	63.31	3.69	—
Transformer only	65.72	64.66	3.54	—
Transformer–context fusion	70.35	68.09	2.69	—
XGBoost backbone	88.49	88.59	1.29	—
+ SHAP/LIME explanations	88.49	88.59	1.29	—
+ Prioritization (θ=0.5) = DEF	**88.97**	**90.09**	**1.29**	**69.7**

Table 10 Auxiliary module performance relative to the XGBoost backbone on the same benchmark; five seeds $\{1, 7, 13, 42, 101\}$, mean ± standard deviation. On CICIoT2023 the sequence and graph inputs are proxies (protocol-group ordering, neighborhood statistics); on UNSW-NB15 they are constructed from true host addresses and timestamps. Sequence and fusion models are evaluated at window level ($n = 7{,}597$ CICIoT2023; $n = 8{,}610$ UNSW-NB15), tabular and context-encoder models at flow level.

Configuration	Macro F1 (%)		% of backbone	
	CICIoT2023	UNSW-NB15	CICIoT2023	UNSW-NB15
XGBoost backbone	88.49 ± 0.13	68.22 ± 0.22	100.0	100.0
Topology-context encoder	41.06 ± 3.29	30.22 ± 0.67	46.4	44.3
LSTM	64.03 ± 0.81	50.49 ± 1.26	72.4	74.0
Transformer	65.72 ± 1.62	58.27 ± 1.52	74.3	**85.4**
Transformer–context fusion	70.35 ± 0.85	59.38 ± 3.15	79.5	**87.0**
Gap to backbone (pts)	**−18.14**	**−8.84**	—	—

nonetheless, on this benchmark both modules *reduce* performance relative to the tabular backbone: a negative result we attribute to the proxy nature of the protocol-group sequences and neighborhood descriptors forced by the absence of host and timestamp fields (Section "Interpretation of detection results"). Fourth, the prioritization layer is where the SOC-facing gains concentrate: it removes roughly half the residual false positives while slightly *improving* macro F1, because suppression falls disproportionately on low-confidence false positives.

Do host and timestamp fields explain the fusion gap?

Section "Interpretation of detection results" attributed the fusion model's shortfall on CICIoT2023 to the absence of host and timestamp identifiers, which forced protocol-group sequences and neighborhood-statistic proxies in place of true per-host histories and a real communication graph. UNSW-NB15 permits that explanation to be tested directly: supplying the identifiers activates genuine per-host sequence construction (37 source-host

sequences ordered by capture time) and a real 5-minute host graph whose node features are out-degree, in-degree, fan-out and slot occupancy.

Because the two benchmarks differ in class cardinality, absolute macro F1 is not comparable across them. We therefore express each auxiliary configuration as a fraction of the *shared* XGBoost backbone on the same dataset, a quantity that is directly comparable.

The results support the explanation given in Section "Interpretation of detection results". The fusion's shortfall relative to the backbone narrows from 18.14 to 8.84 macro-F1 points — from 79.5% to 87.0% of backbone performance — when real host and timestamp fields replace the proxy constructions. Roughly **half (51%) of the observed gap is therefore attributable to the missing identifier fields** rather than to the architecture.

The gain is concentrated in the temporal path. The Transformer alone improves by 11.1 percentage points of relative performance, whereas the topology-context encoder alone is essentially unchanged (46.4% → 44.3%) and the LSTM barely moves (72.4% → 74.0%). Two conclusions follow. First, the benefit derives from genuine event ordering rather than from graph features in isolation, and self-attention exploits that ordering in a way a final-hidden-state recurrent model does not. Second, the fusion significantly outperforms the Transformer alone on CICIoT2023 (paired t-test, $p = 3.2 \times 10^{-3}$) but *not* on UNSW-NB15 ($p = 0.40$), so the added value of graph context over sequence alone is benchmark-dependent and, on the enterprise data, not statistically distinguishable.

A further effect is operationally more consequential than the F1 convergence. On CICIoT2023 the fusion more than doubles the false-positive rate relative to the backbone (2.69% vs. 1.29%, i.e. +1.40 points); on UNSW-NB15 the penalty falls to +0.20 points (1.45% vs. 1.25%). Since analyst workload is driven by false alarms rather than by macro F1, the fusion is close to operationally neutral once real host and timestamp fields are available, whereas on proxy inputs it would have imposed roughly twice the false-alarm burden.

We are explicit about what this does and does not establish. The fusion still does not surpass the tabular backbone on either benchmark (paired t-test, $p = 6.6 \times 10^{-7}$ on CICIoT2023 and $p = 3.8 \times 10^{-3}$ on UNSW-NB15), so the negative result of Section "Component analysis" stands on both datasets and the DEF retains XGBoost as its detection backbone. What the experiment establishes is the *magnitude* of the penalty imposed by absent host and timestamp fields, and hence that temporal–topological modelling requires such fields even to be competitive. Two limitations bound the conclusion. UNSW-NB15 is a 37-source-host testbed, so its communication graph is small and not representative of enterprise-scale topology. Further, because the subset samples 60,889 of 2,059,414 unique records, the density of each 5-minute graph slot is roughly a thirty-fourth of the full corpus, which attenuates degree and fan-out statistics; measured fan-out spans only one to ten distinct destinations per slot. A denser sample, or a dataset with enterprise-scale topology, would test the graph path more severely than we are able to here.

Alert prioritization and SOC impact

Table 11 summarizes SOC workload metrics at the selected operating point $\theta = 0.5$ with weights $(\alpha, \beta, \gamma) = (0.5, 0.4, 0.1)$.

Transfer of the prioritization layer

The prioritization layer is the framework's principal operational contribution, so its behaviour on a second benchmark is a more demanding test than detection accuracy alone. Table 12 compares the layer on both datasets at the common operating point $\theta = 0.5$.

Volume reduction transfers; the precision gain does not. Measured correctly against generated alerts, the layer removes 5.8% of alerts on CICIoT2023 and 2.1% on UNSW-NB15 at the validation-selected operating point. The reduction is modest because most benign traffic is already filtered by the detector itself and never becomes an alert; what the layer removes is a fraction of the *residual* false positives that survive detection. That fraction is larger on CICIoT2023 (91 of 184 false positives) than on UNSW-NB15 (15 of 90), which is the transferable

Table 11 Alert accounting and SOC workload at the operating point θ (selected on validation) on the CICIoT2023 test set. The denominator for the Alert Reduction Rate is the number of alerts the detector *generates* (TP+FP), not the number of flows it inspects: correctly classified benign flows never raise an alert. Of 2,849 generated alerts the prioritization layer suppresses 91 false positives at a cost of 75 true positives, a 5.8% reduction. Precision, recall and F1 are computed over the full test set under Eq. 8.

Metric	Without prioritization	DEF ($\theta = 0.5$)
Flows in test set	9,135	9,135
Alerts generated (TP+FP)	2,849	2,849
true positives (TP)	2,665	—
false positives (FP)	184	—
Alerts presented to analyst	2,849	2,683
Alert Reduction Rate (alerts)	—	**5.8%**
FP suppressed	—	91 of 184
TP suppressed	—	75
Macro precision (%)	88.52	**91.32**
Macro recall (%)	88.52	87.44
Macro F1 (%)	88.43	**89.20**
SHAP explanation time	—	1.6 ms/alert
LIME explanation time	—	144.4 ms/alert

Table 12 Alert prioritization on both benchmarks at $\theta = 0.5$. Metrics after prioritization are computed over the full test set under Eq. 8. Weights are tuned independently on each dataset's validation split under the relative recall floor of Section "Alert prioritization layer".

Metric	CICIoT2023	UNSW-NB15
Selected weights (α, β, γ)	(0.5, 0.4, 0.1)	(0.6, 0.4, 0.0)
Recall floor (98% of val. baseline)	86.9%	67.8%
Alerts generated (TP+FP)	2,849	2,912
Alerts presented	2,683	2,853
Alert Reduction Rate (alerts)	**5.8%**	**2.1%**
FP suppressed	91 of 184	15 of 90
TP suppressed	75	46
Macro recall, unprioritized (%)	88.52	68.64
Macro recall, prioritized (%)	87.44	67.86
change (pts)	−1.08	−0.78
Macro precision, unprioritized (%)	88.52	68.74
Macro precision, prioritized (%)	**91.32**	68.74
change (pts)	**+2.80**	0.00
Macro F1, unprioritized (%)	88.43	67.74
Macro F1, prioritized (%)	**89.20**	67.74

Table 13 Ranking-configuration ablation at the operating point (Editor Comment 4). Each configuration weights the three priority terms as shown; θ is selected on the validation split at a 98%-of-baseline macro-recall floor and applied unchanged to test. **Confidence-only and the full score are statistically indistinguishable** on both benchmarks: adding the SHAP attribution term (β) does not improve ranking. On UNSW-NB15 the SHAP term is in fact mildly anti-correlated with detection confidence (Spearman −0.61), so it perturbs rather than sharpens the ordering.

Configuration	CICIoT2023		UNSW-NB15	
(α, β, γ)	Recall	F1	Recall	F1
Confidence only (1, 0, 0)	87.44	89.20	67.31	67.36
SHAP only (0, 1, 0)	86.64	88.40	67.58	67.64
Context only (0, 0, 1)	88.52	88.43	68.03	67.64
Confidence + SHAP (0.5, 0.5, 0)	87.28	89.12	67.96	67.89
Confidence + context (0.5, 0, 0.5)	88.44	88.43	67.03	67.15
Full score	87.26	89.24	67.86	67.74

finding: the mechanism suppresses low-confidence false positives wherever they exist, but their prevalence, and hence the achievable reduction, is dataset-dependent.

The precision improvement is dataset-dependent: +2.80 points on CICIoT2023 against 0.00 on UNSW-NB15. The mechanism explains why. Suppression raises precision only when the suppressed errors are *low-confidence* false positives, which is what the score $P(a)$ is constructed to identify. On CICIoT2023 the residual errors of the stealthy classes are of exactly that kind. On UNSW-NB15, macro precision is dominated by Analysis, Backdoor and DoS, which are *confidently* misclassified because they overlap Exploits semantically rather than statistically (Section "Cross-dataset generalization"); a confidence-based filter cannot remove a confident error. This is a genuine boundary condition on the method: the prioritization layer reduces analyst volume wherever detection confidence is informative, but it improves precision only where the false positives it removes were themselves uncertain.

The topology term contributes marginally on both datasets. The tuner selected $\gamma = 0.0$ on UNSW-NB15 and $\gamma = 0.1$ on CICIoT2023, so the priority score is driven almost entirely by detection confidence and SHAP attribution magnitude. On UNSW-NB15 the selection was marginal: the configuration (0.5, 0.4, 0.1) reached validation macro recall of 67.72% against a floor of 67.76%, failing by 0.04 points, and its alert reduction (68.89%) was within 0.14 points of the selected configuration. We therefore do not claim that topological context is uninformative, only that its contribution to *ranking* is small under both the protocol-group proxy of CICIoT2023 and the host graph available here. A plausible cause on UNSW-NB15 is sampling density: the subset draws 60,889 of 2,059,414 unique records, so each five-minute graph slot is roughly a thirty-fourth as dense as the full corpus and measured fan-out spans only one to ten distinct destinations (Section "Do host and timestamp fields explain the fusion gap?"). Establishing whether a denser graph would make c_{ctx} a materially useful ranking term is left to future work.

We also note that c_{ctx} is not the same quantity on the two benchmarks. On UNSW-NB15 it is the source host's fan-out within a five-minute window — the structural measure Section "Alert prioritization layer" motivates — whereas on CICIoT2023, which lacks host identifiers, it is the dispersion of local activity within the protocol group. The UNSW-NB15 instantiation is the more faithful of the two, which makes the small selected weight the more informative result.

Suppression is gentler on rare classes than on CICIoT2023. Table 14 gives per-class recall against θ. At the operating point, Fuzzers is the only class to lose recall at all (80.4% → 78.1%); the remaining nine, including the four size-limited families, are unaffected. This contrasts with CICIoT2023, where four classes degrade at $\theta = 0.5$ (Table 17). Most notably Worms — 171 records in the deduplicated corpus and 25 in the test fold — retains its full 68.0% recall up to $\theta = 0.7$, so the rarest class in either benchmark is not the one that pays for the reduction.

Table 14 Per-class recall (%) against θ on UNSW-NB15. At the operating point $\theta = 0.5$ only Fuzzers loses recall relative to the unprioritized backbone. Beyond $\theta = 0.6$, Shellcode and Reconnaissance collapse, bounding the usable range.

Class	θ=0.3	θ=0.4	θ=0.5	θ=0.6	θ=0.7	θ=0.8
Normal	98.6	98.6	98.6	98.8	99.3	99.7
Shellcode	95.6	95.6	95.6	95.1	4.9	0.0
Generic	84.4	84.4	84.4	84.4	84.2	80.9
Reconnaissance	83.7	83.7	83.7	83.7	31.6	0.0
Fuzzers	80.4	80.4	78.1	71.9	50.8	10.2
Worms	68.0	68.0	68.0	68.0	68.0	16.0
Exploits	65.1	65.1	65.1	64.8	62.0	47.7
Analysis	48.3	48.3	48.3	48.3	37.0	18.3
DoS	37.8	37.8	37.8	37.8	35.5	20.2
Backdoor	26.9	26.9	26.9	26.9	23.2	4.0
ARR (%)	54.1	68.0	**68.2**	68.7	77.8	90.4
Macro recall (%)	68.87	68.87	**68.64**	67.98	49.64	29.70

Table 15 Effect of the priority threshold θ on CICIoT2023 test metrics, with the Alert Reduction Rate computed against the 2,849 alerts the detector generates (Editor Comment 1). The operating point is selected on validation, not from this sweep (Section "Alert prioritization layer"); the sweep is shown only to characterize sensitivity. Aggressive thresholds raise precision but suppress genuine attacks, so recall falls steeply beyond the operating region.

θ	Alerts presented	ARR (%)	Recall (%)	Precision (%)	F1 (%)
0.3	2,849	0.0	88.52	88.52	88.43
0.4	2,808	1.4	88.40	89.10	88.60
0.56	**2,683**	**5.8**	**87.44**	**91.32**	**89.20**
0.6	2,500	12.2	85.6	93.8	89.3
0.7	2,176	23.6	78.8	97.3	85.3
0.8	1,627	42.9	62.5	97.5	68.2

Table 16 Sensitivity of the prioritization layer to (α, β, γ), evaluated on the validation set at the screening threshold $\theta = 0.6$ used for weight selection, around the chosen operating point. Perturbing any weight by ± 0.1 changes ARR by under 4 percentage points and recall by under 5, indicating a stable rather than knife-edge configuration.

α	β	γ	ARR (%)	Recall (%)
0.4	0.3	0.3	71.68	85.20
0.4	0.4	0.2	73.17	84.29
0.4	0.5	0.1	74.46	82.34
0.5	0.3	0.2	70.88	86.92
0.5	**0.4**	**0.1**	**72.12**	**85.74**
0.5	0.5	0.0	73.85	83.65
0.6	0.3	0.1	70.68	87.06
0.6	0.4	0.0	71.52	86.35

Table 17 Per-class recall (%) as a function of θ. Suppression cost falls almost entirely on the low-signature classes (Brute Force, Recon, Spoofing, Web Attack), while volumetric attacks retain near-perfect recall up to $\theta = 0.7$. Deployments sensitive to stealthy attacks should operate at $\theta \leq 0.5$ or adopt class-conditional thresholds.

Class	θ=0.3	θ=0.4	θ=0.5	θ=0.6	θ=0.7
Benign	97.1	97.1	98.0	99.4	99.9
Brute Force	78.8	78.5	76.8	68.6	52.3
DDoS	99.0	99.0	99.0	99.0	99.0
DoS	100.0	100.0	100.0	100.0	100.0
Mirai Botnet	100.0	100.0	100.0	100.0	100.0
Recon	75.6	75.6	74.6	72.6	62.5
Spoofing	78.5	78.5	78.5	75.8	72.3
Web Attack	79.3	79.3	77.5	72.6	47.7

The safe operating region is $\theta \leq 0.6$: at $\theta = 0.7$ Shellcode collapses from 95.6% to 4.9% and Reconnaissance from 83.7% to 31.6%, which reinforces the class-conditional threshold recommendation of Section "Discussion".

Threshold and weight sensitivity

Table 15 reports the precision–recall–ARR trade-off across θ with the alert-based denominator, and Table 16 the sensitivity to the weight vector.

Suppression risk for stealthy and rare attack classes

Table 17 answers a question central to deployment: *which* attacks pay for the alert reduction.

XAI explanation quality and efficiency

Table 18 summarizes the explanation modules on both latency and quality axes.

On the tree backbone, SHAP is fast enough (1.6 ms) to be computed for *every* alert — a requirement, since the attribution magnitude feeds the priority score — with no perceptible latency for the analyst. LIME, at 144.4 ms, is generated on demand for the alerts that survive prioritization. For the neural fusion model, gradient-based SHAP is more than two orders of magnitude slower (983.9 ms), which is a further practical argument for the tree backbone in latency-sensitive settings.

SHAP feature importance analysis

Figure 3 presents the SHAP beeswarm plot for the Brute Force decision, and Fig. 4 the global multiclass importance ranking.

LIME local explanation example

Figure 5 and Table 19 show the LIME explanation for an alert the detector actually raises — a flow classified as Mirai Botnet with predicted probability 1.00 — explained against its *predicted* class. Both the figure and the table are generated from a single `explain_instance` call against that class, so the listed conditions and the plotted bars correspond exactly. This illustrates the operational use of local explanations: giving the analyst a small set of directly verifiable protocol-level conditions behind each alert, so that a true positive can be confirmed, or a low-confidence alert dismissed, in seconds.

Runtime and resource profile

The deployed inference path — XGBoost detection followed by TreeExplainer attribution — was profiled over 3,200 test alerts in batches of 32 on commodity CPU hardware, with no GPU. Detection alone requires **0.14 ms** per alert (p95: 0.20 ms) and attribution **3.03 ms** (p95: 4.36 ms), giving a combined **3.17 ms** per alert (p95: 4.49 ms), a sustained throughput of **315 alerts per second**, and a process resident set size of **0.89 GB**. Attribution therefore dominates the latency budget by a factor of roughly twenty. This is the deliberate price of computing a Shapley attribution for *every* alert rather than on demand — a requirement of the DEF design, since the attribution magnitude is a term of the priority score. The 1.6 ms per-instance figure in Table 18 is measured over a single 500-instance batch and reflects the amortized best case; the 3.03 ms figure above reflects the smaller batches characteristic of a streaming deployment and is the operationally relevant number.

In this paper, "real-time" therefore denotes low-single-millisecond per-alert detection-plus-attribution latency at this benchmarked throughput. A capacity of 315 alerts per second corresponds to approximately 1.1 million alerts per hour, four orders of magnitude above the 50–100 alerts per hour that characterize the SOC workload this system targets, so explanation generation imposes no perceptible delay from the analyst's perspective. For comparison, the Transformer–context fusion forward pass costs 0.22 ms per alert, so adopting a neural backbone would shift rather than remove the bottleneck: its GradientExplainer attribution path (983.9 ms, Table 18) is more

Table 18 XAI explanation efficiency and quality. Times are per alert on commodity CPU hardware. Fidelity is the Spearman correlation between mean $|\phi_j|$ and model-agnostic permutation importance; stability is the mean pairwise Jaccard similarity of top-10 feature sets over five repeated explanations of each of 30 instances.

XAI method	Time/alert	Fidelity	Stability
SHAP TreeExplainer (backbone)	**1.6 ms**	0.79	—
SHAP GradientExplainer (fusion)	983.9 ms	—	—
LIME (tabular, 10 features)	144.4 ms	—	0.62

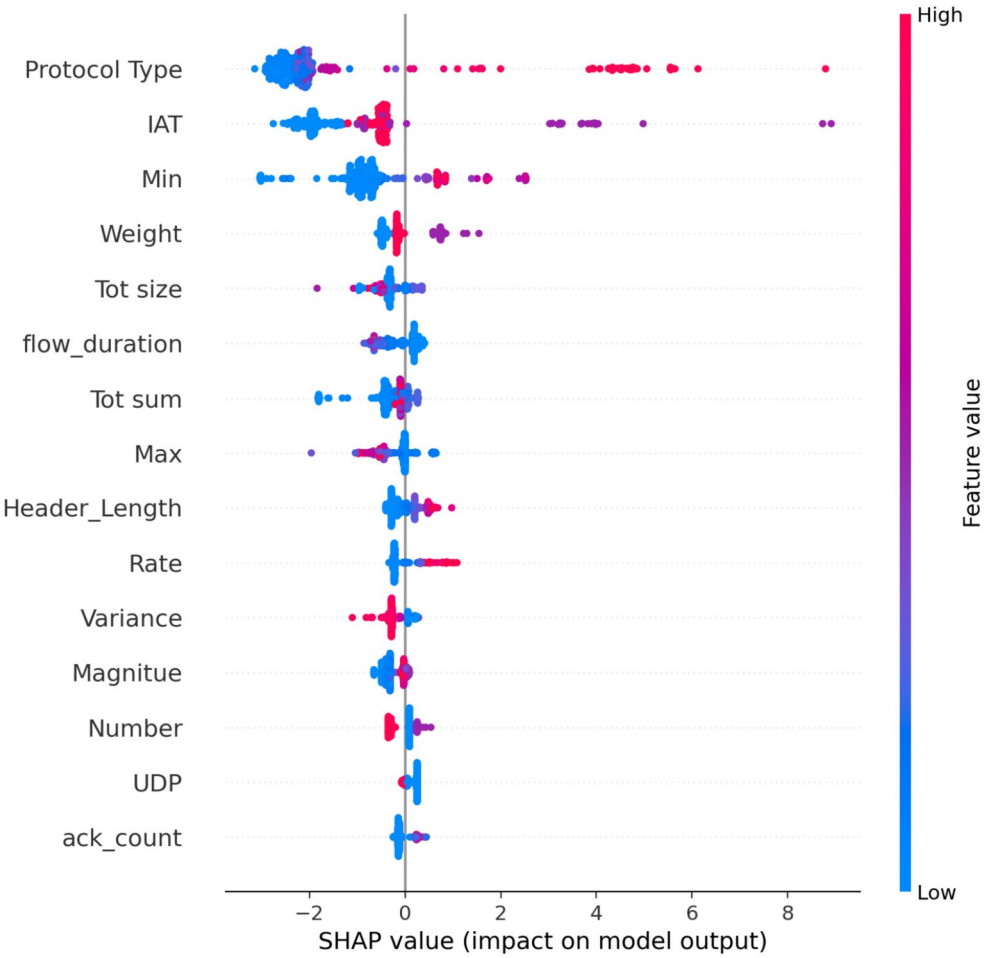


Fig. 3 SHAP beeswarm for the Mirai Botnet vs. rest decision of the XGBoost backbone (500 CICIoT2023 test flows), matching the predicted class of the LIME example in Section "LIME local explanation example". Feature-value colouring indicates the direction in which each feature pushes the Mirai Botnet score.

than two orders of magnitude more expensive than TreeExplainer. These figures are benchmark measurements on a single machine; validation under live SOC load remains future work.

Discussion

Interpretation of detection results

The most instructive finding of this study is the gap between the tabular backbone (88.49% macro F1) and the Transformer–context fusion (70.35%). Three factors explain it. First, the released CICIoT2023 features contain no host addresses or timestamps, so the sequence windows are protocol-group proxies rather than true per-host event histories; the temporal regularities that attention mechanisms exploit in genuine traffic sequences[29] are attenuated. Second, the topology-context encoder substitutes neighborhood statistics for a real communication graph, losing exactly the structural signal (fan-out, lateral movement) that makes GCN-based IDS effective when host identifiers are available[15,16]. Third, the confusion matrices localize the fusion's weakness: DDoS collapses into DoS (0.57 diagonal), a distinction that depends on *source multiplicity* — information that is simply absent without host fields. We report this negative result openly rather than omitting the modules: it delineates the data requirements under which temporal–topological modeling can be expected to pay off, and it justifies the DEF's design decision to build the operational layers (explanation, prioritization) on the strongest available backbone.

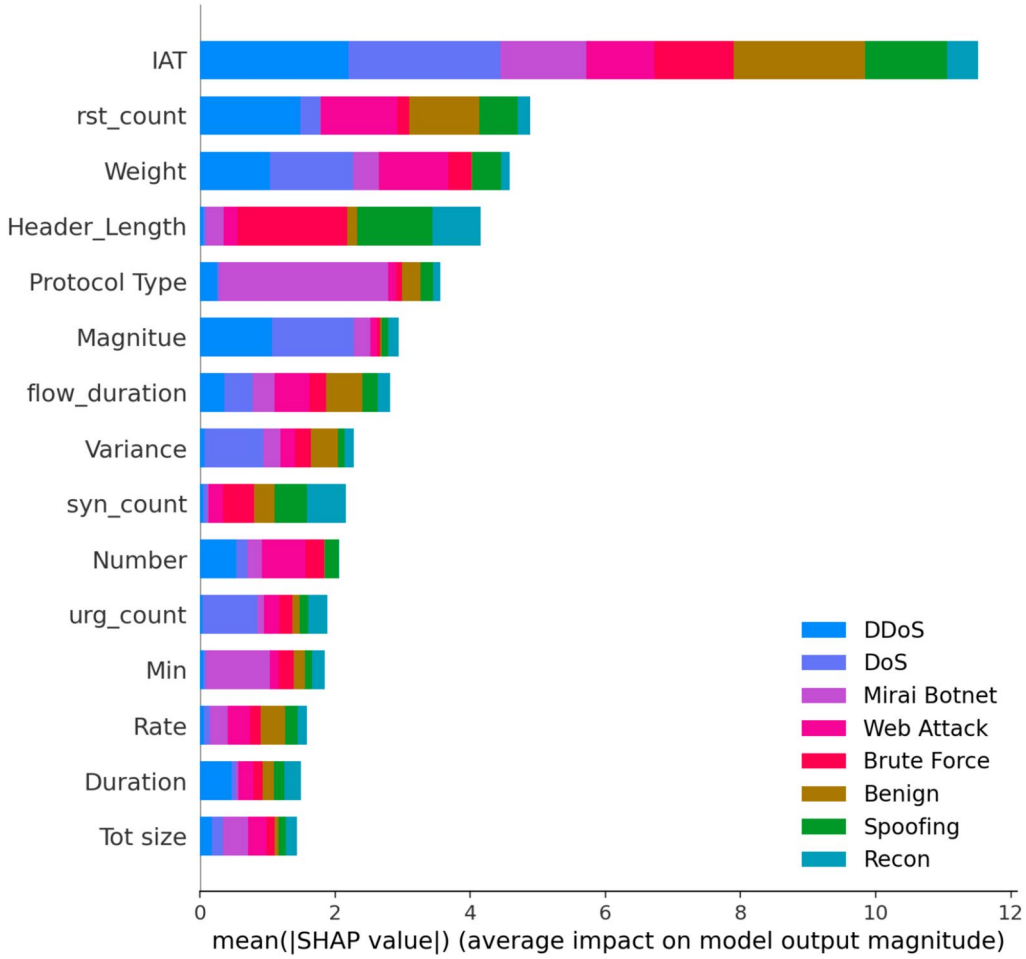


Fig. 4 Global SHAP feature importance across all eight classes. Inter-arrival time (`IAT`), `rst_count`, and `Header_Length` carry the largest overall attribution; `Protocol Type` is dominated by the Mirai Botnet class, reflecting its protocol-specific flooding behaviour.

Section "Do host and timestamp fields explain the fusion gap?" converts that explanation from a conjecture into a measurement. Repeating the identical experiment on UNSW-NB15, whose released records retain host addresses and timestamps, narrows the fusion's shortfall from 18.14 to 8.84 macro-F1 points, so approximately half (51%) of the gap observed on CICIoT2023 is attributable to the missing identifier fields rather than to the architecture. The gain is confined to the temporal path — the Transformer improves by 11.1 percentage points of relative performance while the topology-context encoder and the LSTM barely move — which indicates that genuine event ordering, exploitable by attention but not by a final-hidden-state recurrent model, is the operative ingredient. The fusion's false-alarm penalty also essentially disappears, falling from +1.40 to +0.20 percentage points of FPR relative to the backbone. Accordingly, the fusion components are best understood as an architectural exploration that quantifies those data requirements, not as a route to higher predictive accuracy; the contribution of this work lies at the level of system integration, which Section "Conclusion" states explicitly.

The backbone's own error profile is operationally favourable: volumetric attacks (DDoS 0.99, DoS 1.00, Mirai 1.00 recall) are essentially solved, while the stealthy classes (Brute Force, Recon, Spoofing, Web Attack, 0.76–0.79 recall) remain the open challenge — consistent with their traffic features overlapping legitimate flows. The 1.29% FPR means roughly 1 in 78 benign flows is incorrectly flagged.

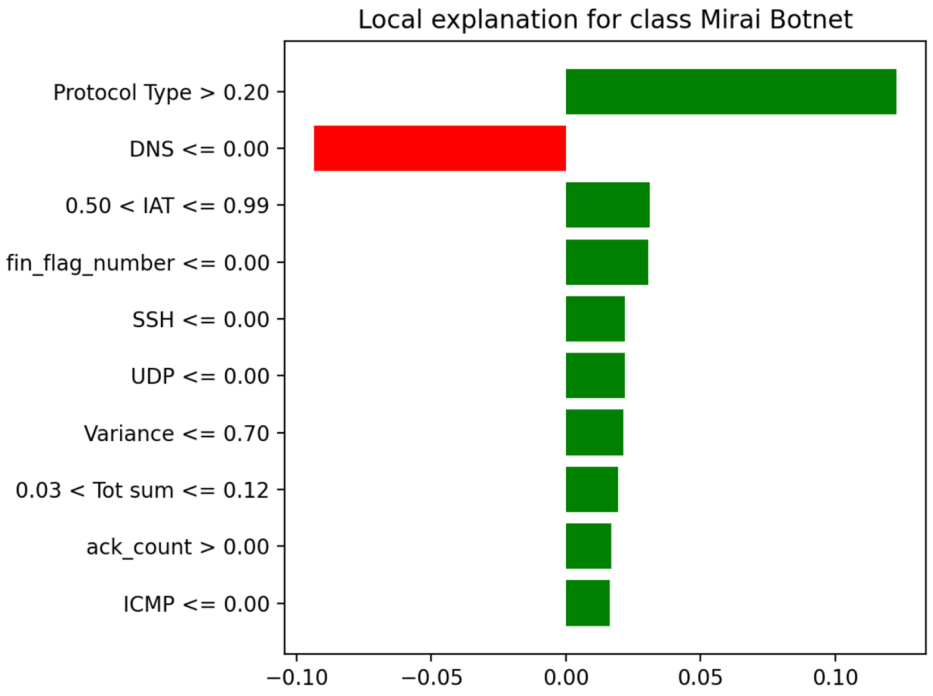


Fig. 5 LIME local explanation for a representative alert scored against the Brute Force class. Red bars oppose the predicted class; green bars support it. The explanation lists directly verifiable protocol-level conditions.

Table 19 LIME top-10 feature contributions for the Mirai Botnet instance in Fig. 5. Positive weight supports the predicted class; negative opposes it. Values are reproduced from `results/lime_table7.csv` of the re-run.

Feature condition	Weight	Direction
SSH ≤ 0.00	−0.435	Opposes Brute Force
Header_Length ≤ 0.00	+0.063	Supports Brute Force
0.50 < IAT ≤ 0.99	−0.038	Opposes Brute Force
DNS ≤ 0.00	+0.038	Supports Brute Force
fin_count > 0.00	−0.035	Opposes Brute Force
flow_duration ≤ 0.00	−0.031	Opposes Brute Force
Protocol Type > 0.20	−0.030	Opposes Brute Force
rst_flag_number ≤ 0.00	+0.029	Supports Brute Force
Variance ≤ 0.70	+0.026	Supports Brute Force
ARP ≤ 0.00	−0.023	Opposes Brute Force

SOC workload impact

Following the editor's guidance, we report the alert reduction against the alerts the detector actually *generates* (TP+FP), not against all inspected flows. Of 2,849 generated alerts on CICIoT2023 the layer removes 5.8% at the validation-selected operating point — specifically 91 of 184 false positives, at a cost of 75 of 2,665 true positives — while raising macro precision by 2.8 points. The practical value of the layer is therefore not a large headline reduction but a targeted removal of low-confidence false positives: it improves the precision of what the analyst sees rather than dramatically shrinking the queue, because the detector has already filtered the benign majority. Any projection to analyst-minutes per hour would require per-alert triage time and staffing figures we did not measure.

Section ""Transfer of the prioritization layer"" shows the mechanism is not an artifact of one benchmark: on UNSW-NB15 the same layer removes a smaller fraction (15 of 90 false positives, 2.1% of generated alerts), because fewer of that benchmark's residual errors are low-confidence. The precision improvement likewise does not transfer, for the same reason, a boundary condition analyzed in Section "Transfer of the prioritization layer".

Table 17 adds the necessary caveat: the recall cost is not uniform. Volumetric attacks are untouched up to $\theta = 0.7$, but Brute Force, Recon, Spoofing, and Web Attack lose 1–8 recall points already at $\theta = 0.5$ and degrade steeply

beyond $\theta = 0.6$. Deployments for which stealthy attacks are the primary concern should operate at $\theta \leq 0.5$ or adopt class-conditional thresholds, which we identify as a concrete extension.

Explanation speed, quality, and analyst workflow

The 1.6 ms TreeExplainer attribution allows SHAP to run on every alert — a prerequisite of the DEF design, since attribution magnitude is a term of the priority score — and the fidelity of 0.79 against permutation importance indicates the attributions track genuine model behaviour rather than artefacts. LIME's 144.4 ms latency comfortably supports on-demand use at post-prioritization alert rates, though its top-10 stability of 0.62 means that on average roughly two of the ten listed conditions differ between repeated explanations of the same instance; analysts should read LIME outputs as indicative evidence rather than a deterministic signature, and the more stable SHAP attributions as the primary rationale. The Mirai Botnet example of Section "LIME local explanation example" shows the dismissal workflow this enables: a single verifiable condition (no SSH activity) suffices to deprioritize the alert. We note that the operational value of this workflow is argued from the properties of the explanations rather than demonstrated with analysts; a usability study is identified as future work in Section "Limitations".

It is worth being explicit about what the fidelity and stability measurements do and do not establish. In the taxonomy of Doshi-Velez and Kim[10], both are *functionally grounded* metrics: fidelity tests whether the attributions track the model's actual decision function, and stability tests whether repeated explanations of the same instance agree. These are necessary conditions for an explanation to be usable — an unfaithful or unstable explanation would actively mislead — but they are not sufficient to establish usefulness to a SOC analyst, which is a human-grounded property: whether explanations shorten triage time, improve decision accuracy, and calibrate rather than inflate trust can only be measured with analysts in the loop. Our claims about the dismissal workflow of Section "LIME local explanation example" are therefore arguments from the properties of the explanations, not demonstrations of analyst benefit; the usability study identified in Section "Limitations" is the appropriate instrument for the latter.

Deployment considerations

The measured profile — 315 alerts/s, 0.89 GB process resident set size, CPU-only — places the DEF within reach of a single commodity server for a mid-size SOC, with no GPU requirement for the backbone path. Architecturally, the DEF sits *behind* the IDS/SIEM as an enrichment and triage layer rather than inline on the packet path, so its latency budget is per-alert, not per-packet; at the alert rates SOCs actually sustain, the measured throughput exceeds requirements by four orders of magnitude. Horizontal scaling is achievable by sharding on traffic partitions. The GradientExplainer path (983.9 ms) is the one component that would require batching or GPU support if a neural backbone were adopted.

Limitations

Five limitations should be acknowledged. First, although the evaluation spans two benchmarks and two traffic domains, both are testbed captures with synthetic background traffic, clean and complete labels, and limited host and asset diversity: CICIoT2023 is an IoT testbed and UNSW-NB15 an enterprise emulation whose traffic originates from only 37 source hosts, which does not approximate enterprise-scale address space, asset heterogeneity, or organizational drift. The absolute figures reported here — including the 5.8% and 2.1% alert-based reduction rates — should therefore not be read as projections for enterprise deployment. What the cross-dataset evidence does support is the weaker claim that the reduction mechanism is not specific to one benchmark: on two independently generated datasets, one IoT and one enterprise, the layer removes low-confidence false positives under one protocol. Operational traffic is noisier and non-stationary and contains attack behaviours absent

from any benchmark, so degradation under distribution shift and zero-day behaviour is expected. Second, the negative fusion result now holds on both benchmarks, and Section "Do host and timestamp fields explain the fusion gap?" attributes roughly half (51%) of the CICIoT2023 gap to absent host and timestamp fields; however, UNSW-NB15 supplies those fields across only 37 source hosts, and because the subset draws 60,889 of 2,059,414 unique records each five-minute graph slot is roughly a thirty-fourth as dense as the full corpus. The graph path therefore remains untested at enterprise scale and under realistic density, and the residual 8.84-point gap cannot be confidently decomposed further. Third, no field deployment or analyst usability study was conducted; claims about triage-time reduction are model-based estimates, and the explanation-quality metrics reported in Section "XAI explanation quality and efficiency" are functionally grounded rather than human-grounded, so they establish faithfulness and repeatability but not analyst usefulness. A user study is future work. Fourth, the prioritization weights and threshold were tuned on this dataset's traffic mix; drift in the benign/attack ratio would require re-calibration, and the validation recall floor of 90% was not attainable (85% was used). Fifth, with five seeds the Wilcoxon signed-rank test cannot fall below $p = 0.0625$; the paired t-tests ($p < 2 \times 10^{-5}$ for all comparisons against the backbone) carry the significance claims.

Conclusion

This paper proposed and evaluated the Dynamic Explainable Framework (DEF) for intrusion detection with inference-time XAI explanations and explanation-aware alert prioritization. Evaluated over five seeded runs on two benchmarks under one protocol — a stratified 60,900-flow CICIoT2023 subset (eight classes) and a deduplicated 60,889-flow UNSW-NB15 subset (ten classes) — the framework achieved:

- **94.05% accuracy** with **88.49% macro F1** at a **1.29% false-positive rate** on CICIoT2023, and **88.26% accuracy** with **68.22% macro F1** at a **1.25% false-positive rate** on UNSW-NB15, using the XGBoost detection backbone. The false-positive rate and AUC-ROC transfer essentially unchanged across domains, and XGBoost outperforms Random Forest on both, so the backbone selection is not an artifact of one benchmark;
- an **alert-based reduction of 5.8%** on CICIoT2023 and **2.1%** on UNSW-NB15 at the validation-selected operating point, computed against the alerts the detector generates (TP+FP) rather than against all inspected flows. The layer removes low-confidence false positives (91 of 184 on CICIoT2023), raising macro precision by 2.8 points where those false positives are uncertain; the precision gain does not transfer to UNSW-NB15, whose residual errors are confident. An explicit ranking ablation shows the detection-confidence term alone accounts for this benefit, with the SHAP-attribution term retained for auditability rather than measurable ranking gain;
- per-alert explanations at **1.6 ms** (SHAP, fidelity 0.79) and **144.4 ms** (LIME, stability 0.62), with a combined detection-plus-attribution latency of 3.17 ms per alert and a CPU throughput of 315 alerts per second;
- a statistically validated negative result on both benchmarks — Transformer and topology-context modules do not surpass the tabular backbone — together with a quantification of its cause: roughly **half** (51%) of the fusion's shortfall on CICIoT2023 is attributable to that benchmark's omission of host and timestamp identifiers, which delineates the data requirements for temporal–topological IDS modeling.

We state the scope of the contribution explicitly. The primary contribution of this work is the unified SOC-oriented pipeline — detection, quantitatively evaluated explanation, and explanation-aware prioritization — together with the prioritization strategy itself, rather than a gain in predictive accuracy over a strong tabular baseline. The Transformer and topology-context components are accordingly best viewed as an architectural exploration that establishes the data conditions under which temporal and topological modeling can be expected to help, not as a guaranteed route to higher accuracy. Similarly, the framework is designed and profiled for real-time operation on benchmark traffic traces; it has not been deployed or validated in a live SOC, and the reported workload reductions are measured on benchmark data rather than observed in operational triage.

Future work will: (i) extend the temporal and graph evaluation to traffic with enterprise-scale topology and full capture density, since UNSW-NB15 supplies genuine host and timestamp fields but only across 37 source hosts and at a thirty-fourth of full slot density; (ii) develop class-conditional prioritization thresholds to protect stealthy attack classes under aggressive alert reduction; (iii) conduct an analyst-centered usability study in an operational SOC to measure actual triage-time effects; and (iv) explore counterfactual explanations that show analysts the minimal traffic changes that would alter an alert classification.

A: Retained feature set

Of the 46 numerical flow features in the released CICIoT2023 feature files, seven were constant across the sampled subset and two were exact duplicates of retained columns; both groups were removed, leaving the 37 features below. Names are reproduced verbatim from the source distribution, including the original spelling of `Magnitue`.

Group	Features
Flow timing (4)	`flow_duration`, `Duration`, `IAT`, `Rate`
Size and statistics (12)	`Header_Length`, `Tot size`, `Tot sum`, `AVG`, `Min`, `Max`, `Std`, `Variance`, `Covariance`, `Magnitue`, `Weight`, `Number`
TCP flag indicators (5)	`syn_flag_number`, `fin_flag_number`, `rst_flag_number`, `psh_flag_number`, `ack_flag_number`
Flag counts (5)	`syn_count`, `fin_count`, `rst_count`, `ack_count`, `urg_count`
Protocol indicators (11)	`Protocol Type`, `TCP`, `UDP`, `ICMP`, `IPv`, `ARP`, `DNS`, `HTTP`, `HTTPS`, `SSH`, `Radius`

For reproducibility we note that the retained features are ordered by descending variance, computed on the pooled subset prior to min–max scaling. The five highest-variance features are `IAT`, `Covariance`, `Header_Length`, `Rate`, and `Tot sum`. This ordering is not a feature-selection step — all 37 survivors are used by every model — but it is functionally significant, because the topology-context encoder of Section "Topology-context encoder" takes the first (highest-variance) feature as its scalar local-activity proxy. The complete ordered list is archived as `data/feature_names.txt` in the code repository.

Author contributions M.U.N.: conceptualization, methodology, software, investigation, formal analysis, visualization, writing — original draft. M.A.: formal analysis and validation of the cross-dataset UNSW-NB15 evaluation, including the deduplication analysis of the source records, the interpretation of the label-taxonomy effect on macro F1, and the analysis of prioritization transfer (Sections "Cross-dataset generalization", "Do host and timestamp fields explain the fusion gap?" and "Transfer of the prioritization layer"); writing — review and editing; funding acquisition. A.N.: supervision, methodology review, writing — review and editing. All authors read and approved the final manuscript and agree to be accountable for the accuracy and integrity of the work.

Funding The researchers would like to thank the Deanship of Graduate Studies and Scientific Research at Qassim University for financial support (QU-APC-2026).

Data availibility Both datasets are publicly available. CICIoT2023 is distributed by the Canadian Institute for Cybersecurity, University of New Brunswick, at https://www.unb.ca/cic/datasets/iotdataset-2023.html; UNSW-NB15 is distributed by the Australian Centre for Cyber Security, UNSW Canberra, at https://research.unsw.edu.au/projects/unsw-nb15-dataset (the four `UNSW-NB15_1..4.csv` source files, which retain host addresses and timestamps, rather than the pre-split training/testing release, which does not). Both processed experimental subsets — 60,900 flows for CICIoT2023 and 60,889 for UNSW-NB15 — together with the sampling seeds, the exact class targets, and the deduplication step required to regenerate them from the official releases, are included in the code repository below.

Code availability All preprocessing, training, explanation, prioritization, and figure-generation code for both benchmarks, the two processed subsets, and the per-seed result ledgers are archived at Zenodo under DOI https://doi.org/10.5281/zenodo.21772509, corresponding to release `v2.2_revision` of the repository maintained at https://github.com/MUsamaNazir/dynamic-xai-intrusion-detection. The archive accompanying the original submission is available at https://doi.org/10.5281/zenodo.21200225.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. IBM Security. *Cost of a Data Breach Report 2023*. IBM Corporation (2023). https://www.ibm.com/reports/data-breach
2. Khraisat, A. & Alazab, A. A critical review of intrusion detection systems in the Internet of Things: techniques, deployment strategy, validation strategy, attacks, public datasets and challenges. *Cybersecurity* **4**(1), 18. https://doi.org/10.1186/s42400-021-00077-7 (2021).
3. Mohale, V. Z. & Obagbuwa, I. C. Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability. *Frontiers in Computer Science* **7**, 1520741. https://doi.org/10.3389/fcomp.2025.1520741 (2025).
4. Gaspar, D., Silva, P. & Silva, C. Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron. *IEEE Access* **12**, 30164–30175. https://doi.org/10.1109/ACCESS.2024.3368377 (2024).
5. Sumathi, S. A scalable and explainable hybrid learning framework for cyber threat detection in software-defined networking environments. *Iranian Journal of Science and Technology — Transactions of Electrical Engineering*. (2026). https://doi.org/10.1007/s40998-026-01129-9
6. Lundberg, S.M. & Lee, S.I. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, **30**, (2017). https://arxiv.org/abs/1705.07874
7. Lundberg, S. M. et al. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence* **2**, 56–67. https://doi.org/10.1038/s42256-019-0138-9 (2020).
8. Ribeiro, M.T., Singh, S., & Guestrin, C. "Why should I trust you?": explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144 (2016). https://doi.org/10.1145/2939672.2939778
9. Barredo Arrieta, A. et al. Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion* **58**, 82–115. https://doi.org/10.1016/j.inffus.2019.12.012 (2020).
10. Doshi-Velez, F. & Kim, B. Towards a rigorous science of interpretable machine learning (2017) arXiv:1702.08608 arXiv preprint.
11. Talukder, M. A. et al. A dependable hybrid machine learning model for network intrusion detection. *Journal of Information Security and Applications* **72**, 103405. https://doi.org/10.1016/j.jisa.2022.103405 (2023).
12. Shone, N., Ngoc, T. N., Phai, V. D. & Shi, Q. A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence* **2**(1), 41–50. https://doi.org/10.1109/TETCI.2017.2772792 (2018).
13. Vaswani, A., et al. Attention is all you need. *Advances in Neural Information Processing Systems*, **30**, (2017). https://arxiv.org/abs/1706.03762
14. Kipf, T.N. & Welling, M. Semi-supervised classification with graph convolutional networks. *Proceedings of the 5th International Conference on Learning Representations (ICLR)* (2017). https://arxiv.org/abs/1609.02907
15. Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M. & Portmann, M. E-GraphSAGE: a graph neural network based intrusion detection system for IoT. *Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS)* https://doi.org/10.1109/NOMS54207.2022.9789878 (2022).
16. Caville, E., Lo, W. W., Layeghy, S. & Portmann, M. Anomal-E: a self-supervised network intrusion detection system based on graph neural networks. *Knowledge-Based Systems* **258**, 110030. https://doi.org/10.1016/j.knosys.2022.110030 (2022).
17. Tavallaee, M., Bagheri, E., Lu, W. & Ghorbani, A. A. A detailed analysis of the KDD CUP 99 data set. *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)* https://doi.org/10.1109/CISDA.2009.5356528 (2009).
18. Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A.A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 108–116 (2018). https://doi.org/10.5220/0006639801080116

19. Neto, E. C. P. et al. CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* **23**(13), 5941. https://doi.org/10.3390/s23135941 (2023).
20. Moustafa, N. & Slay, J. UNSW-NB15: a comprehensive data set for network intrusion detection systems. *Proceedings of the Military Communications and Information Systems Conference (MilCIS)* https://doi.org/10.1109/MilCIS.2015.7348942 (2015).
21. Moustafa, N. & Slay, J. The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Information Security Journal: A Global Perspective* **25**(1–3), 18–31. https://doi.org/10.1080/19393555.2015.1125974 (2016).
22. Debar, H., Dacier, M. & Wespi, A. Towards a taxonomy of intrusion-detection systems. *Computer Networks* **31**(8), 805–822. https://doi.org/10.1016/S1389-1286(98)00017-6 (1999).
23. Breiman, L. Random forests. *Machine Learning* **45**, 5–32. https://doi.org/10.1023/A:1010933404324 (2001).
24. Chen, T. & Guestrin, C. XGBoost: a scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794 (2016). https://doi.org/10.1145/2939672.2939785
25. Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* **16**, 321–357. https://doi.org/10.1613/jair.953 (2002).
26. Cisco Systems. *Snort 3 User Manual* (2024). https://www.snort.org/documents
27. Open Information Security Foundation. *Suricata User Guide* (2024). https://suricata.readthedocs.io
28. Sommer, R. & Paxson, V. Outside the closed world: on using machine learning for network intrusion detection. *Proceedings of the IEEE Symposium on Security and Privacy* 305–316, https://doi.org/10.1109/SP.2010.25 (2010).
29. Alazzam, H., Abualganam, O. & Almobaideen, W. Advanced website fingerprinting for detecting VPN-based censorship evasion: a transformer-based approach. *Scientific Reports* **16**, 16652. https://doi.org/10.1038/s41598-026-41976-4 (2026).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Muhammad Usama Nazir[1] · Mohammed Alsuhaibani[2] · Asri bin Ngadi[1]

✉ Muhammad Usama Nazir
usamanazir@graduate.utm.my

[1] Faculty of Computing, Universiti Teknologi Malaysia, Johor, Malaysia

[2] Department of Computer Science, College of Computer, Qassim University, Buraydah, Saudi Arabia